

TUGAS JURNAL

MODUL 13

2311104004

Marsella dwi julianti

SE07-01

Link Github:

https://github.com/marsellacell/KPL_MARSELLADWIJULIANTI_2311104004_SE0701/tree/marsella/13_DESIGN_PATTERN/modul13_2311104004

Source code dan penjelasan:

B. MENJELASKAN DESIGN PATTERN SINGLETON

A. Dua Contoh Penggunaan Singleton Pattern

- **Logger (sistem pencatatan log)**
Dalam aplikasi besar, kita hanya ingin satu instance logger agar semua bagian program mencatat ke tempat yang sama.
- **Database Connection**
Untuk menghindari membuka koneksi berkali-kali, kita bisa pakai Singleton untuk memastikan hanya ada satu objek koneksi database yang digunakan selama runtime.

B. Langkah-langkah Implementasi Singleton Pattern

1. Buat class biasa, tapi cegah agar tidak bisa diinstansiasi lebih dari satu kali.
2. Gunakan variabel statis (static instance) untuk menyimpan satu-satunya objek.
3. Gunakan method statis (getInstance()) untuk membuat atau mengembalikan instance yang sudah ada.
4. Jangan expose constructor ke publik, cukup melalui getInstance().

C. Tiga Kelebihan dan Kekurangan Singleton

Kelebihan:

1. Kontrol penuh atas objek tunggal – Menghindari konflik jika ada banyak instance.
2. Hemat memori dan resource – Karena hanya dibuat satu kali.
3. Global access point – Mudah diakses dari mana saja tanpa perlu mengoper nilai.

Kekurangan:

1. Sulit di-test (unit test) – Karena instance bersifat global dan state-nya bisa terbawa antar test.
2. Terselubung sebagai global variable – Jika digunakan berlebihan, bisa menimbulkan kode sulit dilacak.

3. Tidak cocok untuk environment multi-threading (di Java misalnya) – Butuh mekanisme khusus untuk menjaga thread safety.

C. Implementasi singleton.js dalam JavaScript

```
1  class Singleton {
2    constructor() {
3      if (Singleton.instance) {
4        return Singleton.instance;
5      }
6
7      this.timestamp = new Date();
8      Singleton.instance = this;
9    }
10
11   getTime() {
12     return this.timestamp;
13   }
14 }
15
16 // MAIN
17 const instance1 = new Singleton();
18 const instance2 = new Singleton();
19
20 console.log("Instance 1:", instance1.getTime());
21 console.log("Instance 2:", instance2.getTime());
22
23 console.log("Apakah sama instance?", instance1 === instance2);
```

Penjelasan baris utama (main):

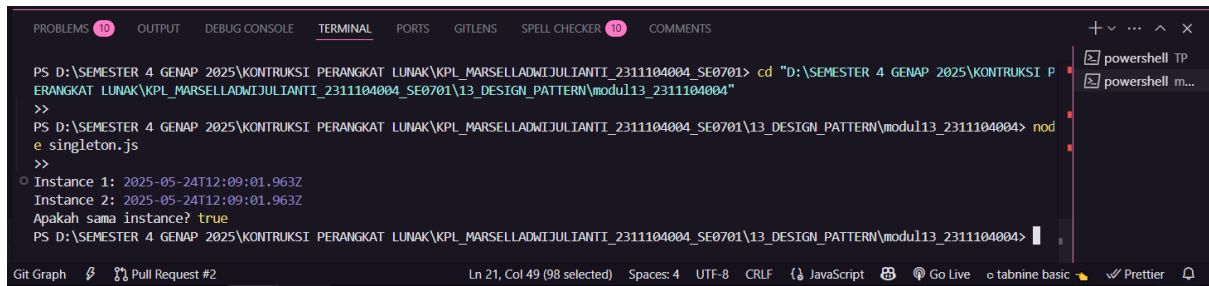
```
1  const instance1 = new Singleton();
2  const instance2 = new Singleton();
```

Pada code diatas meski kelihatannya membuat dua objek, karena Singleton diterapkan, keduanya sebenarnya adalah instance yang sama.

```
1  console.log("Instance 1:", instance1.getTime());
2  console.log("Instance 2:", instance2.getTime());
```

Pada code diatas ini akan mencetak true, membuktikan bahwa hanya **satu objek** yang digunakan.

Output:



```
PS D:\SEMESTER 4 GENAP 2025\KONTRUKSI PERANGKAT LUNAK\KPL_MARSELLADWIJULIANTI_2311104004_SE0701> cd "D:\SEMESTER 4 GENAP 2025\KONTRUKSI PERANGKAT LUNAK\KPL_MARSELLADWIJULIANTI_2311104004_SE0701\13_DESIGN_PATTERN\modul13_2311104004"
>>
PS D:\SEMESTER 4 GENAP 2025\KONTRUKSI PERANGKAT LUNAK\KPL_MARSELLADWIJULIANTI_2311104004_SE0701\13_DESIGN_PATTERN\modul13_2311104004> node singleton.js
>>
Instance 1: 2025-05-24T12:09:01.963Z
Instance 2: 2025-05-24T12:09:01.963Z
Apakah sama instance? true
PS D:\SEMESTER 4 GENAP 2025\KONTRUKSI PERANGKAT LUNAK\KPL_MARSELLADWIJULIANTI_2311104004_SE0701\13_DESIGN_PATTERN\modul13_2311104004>
```